

Lab 03: Inheritance and Polymorphism

Scenario 1: Instead of just having "Books," a modern library has various types of media (E-books, Physical Books, Audiobooks). They share common traits but have unique behaviors.

Instructions:

1. Reuse classes from Lab2 Scenario 1
2. Refactor: Create a Base Class: Rename or refactor Book into a parent class called `LibraryItem`.
 - Common Attributes: title, isbn, isAvailable.
3. Create Subclasses (Inheritance):
 - `PhysicalBook`: Adds an attribute `shelfLocation` (e.g., "A1-04").
 - `EBook`: Adds an attribute `downloadUrl` and `fileSize`.
4. Implement Polymorphism:
 - In `LibraryItem`, create a method `displayDetails()`.
 - Override this method in both `PhysicalBook` and `EBook` to print their unique info (e.g., the E-book should print its URL, and the Physical Book should print its shelf location).
5. Test with a List: Create a list of `LibraryItem` that contains both a `PhysicalBook` and an `EBook`. Loop through the list and call `displayDetails()` on each.
6. Challenge:
 - The Power of the Parent Class (Polymorphism): If we have a list of type `LibraryItem[]`, we can store both `PhysicalBook` and `EBook` objects in it. When we call `item.displayDetails()`, Java knows exactly which version of the method to run. This is this better than creating two separate lists (one for physical books and one for e-books).

```
List<LibraryItem> allItems = new ArrayList<>();
allItems.addAll(...);
for (LibraryItem item : allItems) { item.displayDetails(); }
```

- Specialized Behavior: Imagine the library adds a 'Late Fee' system. For `PhysicalBook`, the fee is 5 Baht per day. For `EBook`, there are no late fees because the file automatically expires. How would you use Method Overriding to implement a `calculateLateFee(int days)` method?
7. Submit the upgrade code and sample output

Main code

```
package com.library.lab03;

import java.util.ArrayList;
import java.util.List;

public class LibraryManagementApp {
    public static void main(String[] args) {
        System.out.println("\n" + "=".repeat(60));
        System.out.println("  LIBRARY MANAGEMENT SYSTEM - POLYMORPHISM DEMO");
        System.out.println("=".repeat(60));
        // Create a list of LibraryItem (demonstrates polymorphism)
        List<PhysicalBook> physicalBooks = new ArrayList<>();
        List<EBook> eBooks = new ArrayList<>();
        // Add Physical Books
        physicalBooks.add(new PhysicalBook("Java Programming", "John Smith", "978-0134685991",
450.0, "A1-04"));
        physicalBooks.add(new PhysicalBook("Clean Code", "Robert Martin", "978-0132350884", 520.0,
"B2-15"));
        physicalBooks.add(new PhysicalBook("Design Patterns", "Gang of Four", "978-0201633612",
680.0, "A3-22"));
        // Add E-Books
        eBooks.add(new EBook("Effective Java", "Joshua Bloch", "978-0134685991",
            "https://library.ebooks.com/effective-java.pdf", 5.2));
        eBooks.add(new EBook("Python Crash Course", "Eric Matthes", "978-1593279288",
            "https://library.ebooks.com/python-crash.pdf", 8.7));
        // Instantiate member object
        Member member1 = new Member("M001", "Somsak");
        Member member2 = new Member("M002", "Suda");
        // Display all items using polymorphism
        System.out.println("\n--- ALL LIBRARY ITEMS (Polymorphism Demo) ---");
        System.out.println("Calling displayDetails() on each item in the list:");
        System.out.println();
        for (PhysicalBook item : physicalBooks) {
            item.displayDetails();
        }
        for (EBook item : eBooks) {
            item.displayDetails();
        }
        // Demonstrate checkOut() method
        System.out.println("\n--- TESTING CHECKOUT FUNCTIONALITY ---");
        System.out.println("\nMember Somsak borrows Physical Book:");
        physicalBooks.get(0).checkOut(member1);
        System.out.println("\nMember Suda borrows E-Book:");
```

```

eBooks.get(1).checkOut(member2);
System.out.println("\nAttempting to checkout an already borrowed item:");
physicalBooks.get(0).checkOut(member2);
// Display items after checkout using polymorphism
System.out.println("\n--- ITEMS STATUS AFTER CHECKOUT ---");
for (PhysicalBook item : physicalBooks) {
    item.printSummary();
}
// Demonstrate returnItem() method
System.out.println("\n--- TESTING RETURN FUNCTIONALITY ---");
System.out.println("\nReturning Physical Book:");
physicalBooks.get(0).returnItem();
// Test borrowing limit
System.out.println("\n--- TESTING BORROW LIMIT ---");
physicalBooks.get(0).checkOut(member1);
physicalBooks.get(1).checkOut(member1);
physicalBooks.get(2).checkOut(member1);
eBooks.get(1).checkOut(member1); // Should be denied (limit reached)
// Final display using polymorphism
System.out.println("\n--- FINAL LIBRARY STATUS (Polymorphism Demo) ---");
for (PhysicalBook item : physicalBooks) {
    item.displayDetails();
}
for (EBook item : eBooks) {
    item.displayDetails();
}
// ===== METHOD OVERRIDING: Late Fee Demo =====
System.out.println("\n" + "=".repeat(60));
System.out.println("  METHOD OVERRIDING: LATE FEE CALCULATION DEMO");
System.out.println("=".repeat(60));
// Simulate late returns
int daysLate = 5;
System.out.println("\n--- Late Fee Calculation (" + daysLate + " days late) ---");
System.out.println("\nPhysical Books (5 Baht per day late fee):");
for (PhysicalBook book : physicalBooks) {
    double lateFee = book.calculateLateFee(daysLate);
    System.out.printf(" 📖 %s: %.2f Baht\n", book.getTitle(), lateFee);
}
System.out.println("\nE-Books (NO late fees - files auto-expire):");
for (EBook book : eBooks) {
    double lateFee = book.calculateLateFee(daysLate);
    System.out.printf(" 📖 %s: %.2f Baht\n", book.getTitle(), lateFee);
}
}
}

```

Example Output:

```
=====
LIBRARY MANAGEMENT SYSTEM - POLYMORPHISM DEMO
=====

--- ALL LIBRARY ITEMS (Polymorphism Demo) ---
Calling displayDetails() on each item in the list:

📖 PHYSICAL BOOK
- Title:           Java Programming
- Author:          John Smith
- ISBN:            978-0134685991
- Price:           450.0 Baht
- Shelf Location:  A1-04
- Status:          Available
- Return Due Date: N/A (Book is available)

📖 PHYSICAL BOOK
- Title:           Clean Code
- Author:          Robert Martin
- ISBN:            978-0132350884
- Price:           520.0 Baht
- Shelf Location:  B2-15
- Status:          Available
- Return Due Date: N/A (Book is available)

📖 PHYSICAL BOOK
- Title:           Design Patterns
- Author:          Gang of Four
- ISBN:            978-0201633612
- Price:           680.0 Baht
- Shelf Location:  A3-22
- Status:          Available
- Return Due Date: N/A (Book is available)

📖 E-BOOK
- Title:           Effective Java
- Author:          Joshua Bloch
- ISBN:            978-0134685991
- Download URL:    https://library.ebooks.com/effective-java.pdf
- File Size:       5.2 MB
- Status:          Available
- Return Due Date: N/A (E-book is available)

📖 E-BOOK
- Title:           Python Crash Course
- Author:          Eric Matthes
- ISBN:            978-1593279288
- Download URL:    https://library.ebooks.com/python-crash.pdf
- File Size:       8.7 MB
- Status:          Available
- Return Due Date: N/A (E-book is available)
```

--- TESTING CHECKOUT FUNCTIONALITY ---

Member Somsak borrows Physical Book:

Item 'Java Programming' has been checked out successfully.

Item 'Java Programming' has been borrowed by Somsak.

Return Due Date: 2026-02-04

Member Suda borrows E-Book:

Item 'Python Crash Course' has been checked out successfully.

Item 'Python Crash Course' has been borrowed by Suda.

Return Due Date: 2026-02-04

Attempting to checkout an already borrowed item:

Error: Item 'Java Programming' is already borrowed and cannot be checked out again.

--- ITEMS STATUS AFTER CHECKOUT ---

PhysicalBook[Title='Java Programming', Location='A1-04', Status='Borrowed']

PhysicalBook[Title='Clean Code', Location='B2-15', Status='Available']

PhysicalBook[Title='Design Patterns', Location='A3-22', Status='Available']

--- TESTING RETURN FUNCTIONALITY ---

Returning Physical Book:

Item 'Java Programming' has been returned successfully.

--- TESTING BORROW LIMIT ---

Item 'Java Programming' has been checked out successfully.

Item 'Java Programming' has been borrowed by Somsak.

Return Due Date: 2026-02-04

Item 'Clean Code' has been checked out successfully.

Item 'Clean Code' has been borrowed by Somsak.

Return Due Date: 2026-02-04

Item 'Design Patterns' has been checked out successfully.

Item 'Design Patterns' has been borrowed by Somsak.

Return Due Date: 2026-02-04

Error: Item 'Python Crash Course' is already borrowed and cannot be checked out again.

--- FINAL LIBRARY STATUS (Polymorphism Demo) ---

 PHYSICAL BOOK

- Title:	Java Programming
- Author:	John Smith
- ISBN:	978-0134685991
- Price:	450.0 Baht
- Shelf Location:	A1-04
- Status:	Borrowed
- Return Due Date:	2026-02-04

 PHYSICAL BOOK

- Title:	Clean Code
- Author:	Robert Martin
- ISBN:	978-0132350884

- Price: 520.0 Baht
- Shelf Location: B2-15
- Status: Borrowed
- Return Due Date: 2026-02-04

 PHYSICAL BOOK

- Title: Design Patterns
- Author: Gang of Four
- ISBN: 978-0201633612
- Price: 680.0 Baht
- Shelf Location: A3-22
- Status: Borrowed
- Return Due Date: 2026-02-04

 E-BOOK

- Title: Effective Java
- Author: Joshua Bloch
- ISBN: 978-0134685991
- Download URL: <https://library.ebooks.com/effective-java.pdf>
- File Size: 5.2 MB
- Status: Available
- Return Due Date: N/A (E-book is available)

 E-BOOK

- Title: Python Crash Course
- Author: Eric Matthes
- ISBN: 978-1593279288
- Download URL: <https://library.ebooks.com/python-crash.pdf>
- File Size: 8.7 MB
- Status: Borrowed
- Return Due Date: 2026-02-04

=====

METHOD OVERRIDING: LATE FEE CALCULATION DEMO

=====

--- Late Fee Calculation (5 days late) ---

Physical Books (5 Baht per day late fee):

 Java Programming: 25.00 Baht
 Clean Code: 25.00 Baht
 Design Patterns: 25.00 Baht

E-Books (NO late fees - files auto-expire):

 Effective Java: 0.00 Baht
 Python Crash Course: 0.00 Baht

Scenario 2: Using a "Parent Class" as a blueprint and allowing "Subclasses" to create unique behaviors. This allows the system to support many types of characters without changing the core game engine logic.

Instructions:

1. Reuse classes from Lab2 Scenario 2

2. Create Subclasses (Inheritance): Create classes that inherit (extends) from the Character class:

- Warrior: Add an attribute armorValue. Override the attack() method to increase damage by 1.5x.
- Mage: Add an attribute mana. Override the attack() method to perform a "Magic Missile" spell.

3. Implement Polymorphism (Method Overriding):

- In the parent class Character, create a method receiveDamage(int amount) to reduce HP.
- In the Warrior subclass, Override receiveDamage. The logic should subtract the armorValue from the incoming damage before reducing HP (e.g., if damage is 20 and armor is 5, HP only drops by 15).

4. The Battlefield (Polymorphic List):

- Create an ArrayList<Character> team = new ArrayList<>();.
- Add both Warrior and Mage objects into this single list.
- Use a loop to make every character in the list attack() the same enemy to see the different results.

5. Challenge:

- Flexibility of the Parent Type: add a new class called Archer, don't change the 'Battle Logic' code we wrote for the ArrayList. This is the benefit of referencing characters using the Parent Class type (Character) even when the actual object is a Subclass.
- Utilizing the Superclass (The super Keyword): In the Warrior.receiveDamage() method, if we want to keep the original HP reduction logic but only add the armor calculation, how should we use the super keyword to avoid rewriting the HP subtraction code from the parent class?

6. Submit the upgrade code and sample output

Main class

```
package com.rpg.lab03;

import java.util.*;

public class RPGGameApp {

    public static void main(String[] args) {
        System.out.println("\n" + "=".repeat(70));
        System.out.println("  SCENARIO 3: RPG BATTLEFIELD - POLYMORPHISM & METHOD OVERRIDING");
        System.out.println("=".repeat(70));
        // ===== CREATE WEAPONS =====
        System.out.println("\n[STEP 1] Creating Weapons...");
        Weapon swordWeapon = new Weapon("Excalibur", "Melee", 30, "Slash Attack");
        Weapon staffWeapon = new Weapon("Staff of Mysticism", "Magic", 20, "Spell Cast");
        Weapon bowWeapon = new Weapon("Bow of Elven Make", "Ranged", 18, "Precise Shot");
        // ===== CREATE TEAM (POLYMORPHIC LIST) =====
        System.out.println("\n[STEP 2] Creating Battlefield Team...");
        List<Character> team = new ArrayList<>();
        // Add Warrior
        Warrior warrior = new Warrior("Aragorn", 12, 2000, 25, 8, 15, swordWeapon);
        team.add(warrior);
        // Add Mage
        Mage mage = new Mage("Gandalf", 14, 1200, 20, 5, 150, staffWeapon);
        team.add(mage);
        // Add Archer
        Archer archer = new Archer("Legolas", 13, 1400, 22, 6, 85, bowWeapon);
        team.add(archer);
        // Create an enemy
        Character enemy = new Character("Dragon", 15, 2500, 30, 10, new Weapon("Claws", "Melee",
25, "Bite"), "Boss");
        // ===== DISPLAY INITIAL STATUS =====
        System.out.println("\n[STEP 3] Initial Team Status:");
        for (Character member : team) {
            member.displayCharacterDetails();
        }
        System.out.println("\n--- ENEMY ---");
        enemy.displayCharacterDetails();
        // ===== POLYMORPHIC BATTLEFIELD: All Attack Same Enemy =====
        // =====
        System.out.println("\n" + "=".repeat(70));
        System.out.println("  [STEP 4] POLYMORPHIC COMBAT: Team Attacks Enemy");
        System.out.println("  Same method call (attack), DIFFERENT behavior for each type!");
        System.out.println("=".repeat(70));
    }
}
```



```

// Loop through team and have each character attack the enemy
for (Character member : team) {
    System.out.println("\n--- " + member.getName() + "'s Turn ---");
    // This demonstrates POLYMORPHISM - same method, different implementations
    if (member instanceof Warrior) {
        System.out.println("(Using Warrior.attack() - 1.5x damage bonus)");
    } else if (member instanceof Mage) {
        System.out.println("(Using Mage.attack() - Magic Missile spell)");
    } else if (member instanceof Archer) {
        System.out.println("(Using Archer.attack() - Ranged arrow attack with accuracy)");
    }
    member.attack(enemy);
}

// ===== SHOW ENEMY STATUS AFTER ATTACKS =====
System.out.println("\n\n[STEP 5] Enemy Status After Team Attacks:");
enemy.displayCharacterDetails();

// ===== ENEMY COUNTERATTACK =====
System.out.println("\n[STEP 6] Enemy Counterattacks!");
System.out.println("\n--- Dragon attacks Warrior ---");
enemy.attack(warrior);
System.out.println("\n--- Dragon attacks Mage ---");
enemy.attack(mage);
System.out.println("\n--- Dragon attacks Archer ---");
enemy.attack(archer);

// ===== FINAL STATUS =====
System.out.println("\n[STEP 7] Final Team Status:");
for (Character member : team) {
    member.displayCharacterDetails();
}
}
}

```

Example Output:

```
✂ Raw Attack Damage: 55
🛡 Gandalf's Defense: 5 (reduces damage from 55 to 50)
💥 Actual Damage Taken: 50
❤️ Gandalf's HP: 1150/1200

--- Dragon attacks Archer ---
Dragon (Boss) attacks Legolas with Claws!
✂ Raw Attack Damage: 55
🛡 Legolas's Defense: 6 (reduces damage from 55 to 49)
💥 Actual Damage Taken: 49
❤️ Legolas's HP: 1351/1400

[STEP 7] Final Team Status:

--- ARAGORN (WARRIOR) ---
Status: Active
Level: 12
Health Points: 1968/2000
Damage: 25
Defense: 8
Armor Value: 15
Weapon: Excalibur (Type: Melee, Damage: 30, Ability: Slash Attack)

--- GANDALF (MAGE) ---
Status: Active
Level: 14
Health Points: 1150/1200
Mana: 120/150
Damage: 20
Defense: 5
Weapon: Staff of Mysticism (Type: Magic, Damage: 20, Ability: Spell Cast)

--- LEGOLAS (ARCHER) ---
Status: Active
Level: 13
Health Points: 1351/1400
Damage: 22
Defense: 6
Accuracy: 85%
Range Advantage: Yes (30% bonus)
Weapon: Bow of Elven Make (Type: Ranged, Damage: 18, Ability: Precise Shot)
```